Household Services Application

By Vanika Dangi

DS23F1000071 | Modern Application Development – I | Sept, 2024 Term

AUTHOR

Name: Vanika Dangi

Roll No: DS23F1000071

Email: 23f1000071@ds.study.iitm.ac.in

About Me:

I am a data science student with a strong foundation in mathematics, statistics, and physics, which fuels my analytical and problem-solving skills. I am proficient in programming languages like Python, Java, HTML, CSS, and JavaScript, and enjoy building practical solutions through coding. Alongside my technical pursuits, I write blogs on Medium, exploring a variety of topics, including physics, to share knowledge and inspire curiosity.

DESCRIPTION OF PROJECT

The Household Services Application is a multi-user platform designed to manage household services efficiently. It includes three user roles: Admin, Professional, and Customer. The admin oversees the system, handling professional approvals, managing users, and maintaining service-related data. Professionals sign up to offer one service, and their applications require admin approval. Customers register to book available services. The system also features visual analytics, like bar and pie charts, to display ratings and service requests, providing actionable insights for stakeholders.

TECHNOLOGIES USED

Flask: Backend framework for building the web application.

Flask-SQLAlchemy: ORM (Object-Relational Mapping) tool for database interactions.

SQLite: Database management system for storing application data.

HTML/CSS/JavaScript: Frontend technologies for user interface design and interactivity.

Chart.js: JavaScript library for creating interactive data visualizations (e.g., bar and pie charts).

Flask-Login: Extension for managing user sessions and authentication.

Python-dotenv: For managing environment variables securely.

Jinja2: Template engine for rendering HTML pages dynamically.

Werkzeug: Utility library for WSGI applications.

Supporting Libraries: Blinker, SQLAlchemy, and others to enhance functionality and performance.

ARCHITECTURE

The application is modularly designed to ensure clear separation of concerns and ease of maintenance. The main application code resides in app.py, and additional components are organized as follows:

routes.py: Defines all the application routes and maps them to corresponding functionalities for customers, professionals, and admins.

models.py: Contains database models and relationships for the application's entities like users, services, and service requests.

config.py: Manages the application's configuration settings (e.g., database URI, secret keys, and debug flags).

templates/: Contains all the HTML files used to render dynamic web pages for different functionalities and user roles.

static/: Stores static assets like CSS, and media files.

instance/: Holds application-specific data like the SQLite database file (db.sqlite3).

requirements.txt: Lists all dependencies required to run the application.

FEATURES

User Management

Registration & Authentication: Allows customers and professionals to register, with a separate signup process for each type of user. Professionals must be approved by the admin before they can log in.

Role-Based Access Control: Three distinct roles — Admin, Customer, and Professional — each with different permissions. Admins manage all aspects of the platform, professionals can offer services, and customers can book services.

Profile Management: Both customers and professionals can manage their profiles to update personal information such as email, addresses, experience, description.

Service Management

Service Catalog & Categorization: Services are categorized under various types, and professionals can list their offerings in specific categories.

Service Listings: Customers can browse the catalog of services and request bookings for the services they need.

Service Offering by Professionals: Each professional can offer only one service but can update or modify it as required.

Service Requests & Status

Request Management: Customers can create service requests, which professionals can accept or reject based on availability.

Automatic Static Updates: The system automatically updates status after service completion, with an option for customers to close and rate the service.

Service Status Tracking: Customers and professionals can track service request progress, including status updates, through the application.

Admin Dashboard

Professional Approval Workflow: Admins review and approve professional registrations before they can log in and offer services.

Transaction & Request Monitoring: Admins can monitor all service requests and their status across the platform.

Notifications: Admins are notified about rejected professionals, ensuring smooth communication with users.

Feedback & Ratings

Service Feedback: Customers can provide feedback on the service received, allowing professionals to improve their offerings.

Ratings System: Professionals are rated out of 5 stars, and the feedback system helps assess service quality and customer satisfaction.

DB SCHEMA

1. User

Columns:

id (PK): Unique ID.

username (Unique), email (Unique), role (admin, customer, professional).

status (active, blocked), is_verified.

2. Service

Columns:

id (PK): Unique ID.

name, description, price.

category_id (FK), professional_id (FK, one-to-one).

3. Category

Columns:

id (PK): Unique ID.

name: Category name.

4. Transaction

Columns:

id (PK): Unique ID.

customer_id (FK), service_request_id (FK).

amount, status, timestamp.

5. Feedback

Columns:

id (PK): Unique ID.

service_request_id (FK), rating (1-5), remarks, timestamp.

6. Service Requests

Columns:

id (PK): Unique ID.

customer_id (FK), professional_id (FK), service_id (FK).

status (pending, accepted, completed), timestamp.

Key Relationships

User ↔ Service Requests:

One customer/professional ↔ multiple requests.

Service ↔ Category:

Many services ↔ one category.

Service Requests ↔ Feedback:

One request ↔ one feedback.

Service Requests ↔ Transaction:

One request ↔ one transaction.

Real-Time Chat System:

Add a Messages table:

Columns: id, service_request_id, sender_id, receiver_id, message, timestamp.

Advanced Search and Filtering:

Payment Gateway Integration:

Extend the Transaction table to include:

payment_gateway: Name of the payment gateway used.

transaction_reference: External transaction ID.

Professional Dashboard Enhancements:

VIDEO LINK

A video demonstration of the project is available here.

https://drive.google.com/file/d/1dwWV9Olum-Ghlm8QvloOVWd49J2LswU6/view?usp=sharing

THANK YOU